

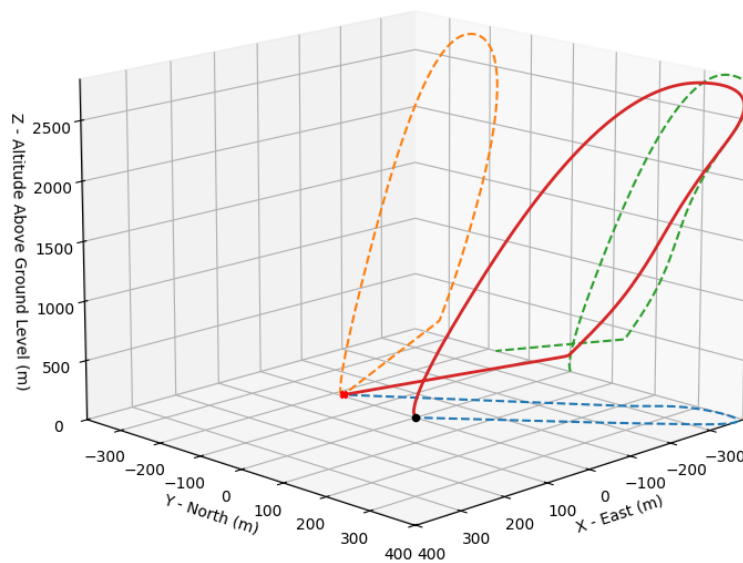


UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



FLIGHT ANALYSIS OF THE SKYBREAKER ROCKET

Flight Trajectory



Author:

Aitor Pérez Grau

Course:

1º Ingeniería Aeroespacial

Entity:

FARADAY ROCKETRY UPV

Date:

14/02/2026

INDEX

1. Introduction.....	3
2. Flight analysis	5
2.1 Analysis with a heading of 300°	5
2.1.1 Parameters obtained.....	5
2.2 Analysis with a heading angle such that it maximizes the apogee.....	9
2.2.1 Reasoning for maximizing the apogee	9
2.2.2 Parameters obtained with said heading	11

1. Introduction

This document contains the flight analysis of the Skybreaker rocket developed by Faraday Rocketry UPV, although without the aerodynamic element of the airbrake that this model carries, and fictitiously with the launch date of February 15, 2026 at 1 CET at the INTA (National Institute of Aerospace Technology) located in Huelva, Spain. For this, the RocketPy library implemented in Python has been used, which allows for highly accurate 6-degrees of freedom simulations.

This analysis represents the culmination of 4 months of daily work, with an average of one hour per day of extremely enriching learning, first of the Python programming language and, later, of the RocketPy library. Next, I will explain exactly what skills have been acquired.

Firstly, I have managed to understand and implement Python concepts such as operators, loops, functions, lambda (anonymous) functions, generators, basic iterables, exception handling, the object-oriented programming paradigm including attributes, methods, classes, instances, inheritances, constructors, polymorphisms, or encapsulation. Also modules, packages, redistributable packages; handling external files with 'a,w,r' modes and their extensions with '+', and pointer behavior in them; graphical interfaces with the Tkinter library with different widgets such as root, frame, label, entry, button, text, scrollbar, radio buttons, check buttons, and menus, and obtaining information from these widgets through smart variables and their internal operation from Tcl/Tk, as well as pop-up windows.

Likewise, I have also become familiar with data processing in databases with SQLite (creating, handling, updating, and closing a database, using SQL instructions and its visualization with tools like DB Browser) and its general internal operation; with the relational programming paradigm with functions like filter or map; with regular expressions, decorators, and their internal operation, and with documentation using docstrings and its use for testing with the doctest module. Finally, I have learned to create data and perform calculations with the NumPy library and to create graphs, bar charts, and 2D and 3D histograms with Matplotlib, also utilizing some of the multiple parameters they accept.

On the other hand, regarding RocketPy, in terms of creating the simulation, I have learned to perform a basic flight simulation. For this, it is necessary to instantiate the Environment class and, also, determine the launch date using the datetime library and the atmospheric model, create an object of one of the subclasses of the Motor class (LiquidMotor, SolidMotor, HybridMotor) that will be added to the instance of the Rocket class also created. Additionally, aerodynamic elements such as fins, structural elements like the nose cone, and recovery elements like parachutes are required, added through methods of the Rocket class, each also being a class

instance. Finally, to carry out the simulation, the Flight class is instantiated, which will contain the simulation data. Furthermore, I have become familiar with the rocket and motor coordinate axes, including where the axes are located, possible orientations, and the origins of the system.

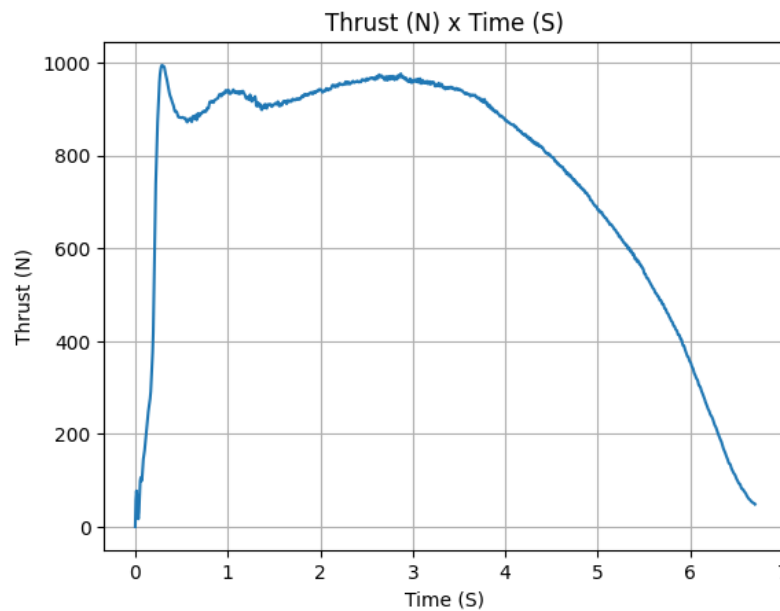
In terms of the data, I have learned and assimilated the methods of the `_FlightPrints` and `_FlightPlots` classes, which are called through the `prints` and `plots` attributes of the `Flight` class, along with other specific methods and attributes used to obtain the data presented in the document, such as `total_impulse`, `burn_duration` from the `SolidMotor` class, or `center_of_mass()`, `cp_position()`... from the `Rocket` class. Likewise, I have investigated the documentation and the internal code of the library in order to know its internal structure and understand the relationships between its elements.

2. Flight analysis

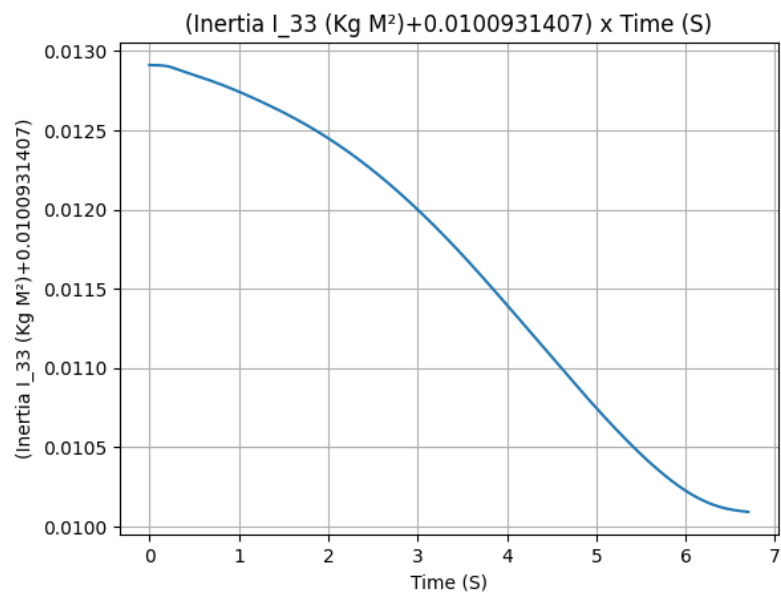
2.1 Analysis with a heading of 300°

2.1.1 Parameters obtained

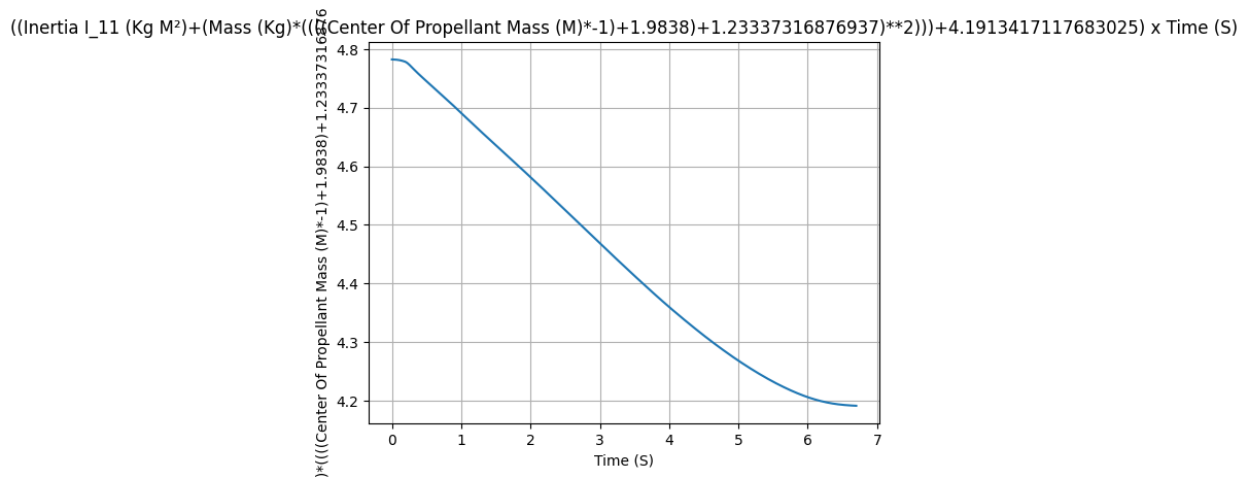
- Maximum thrust: 994.88 N
- Total impulse: 5032.54 Ns
- Burn time: 6.6992 s
- Thrust curve as a function of time



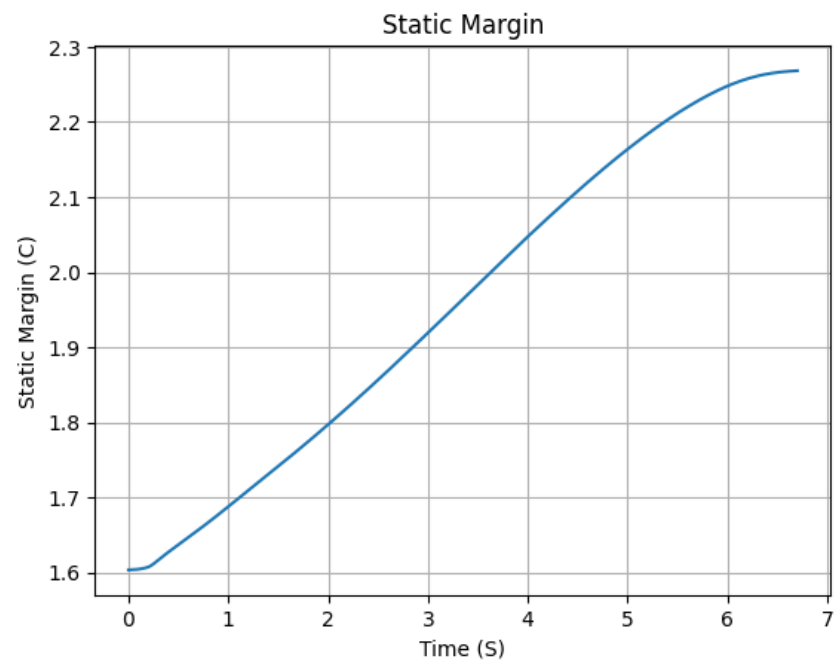
- Longitudinal inertia as a function of time



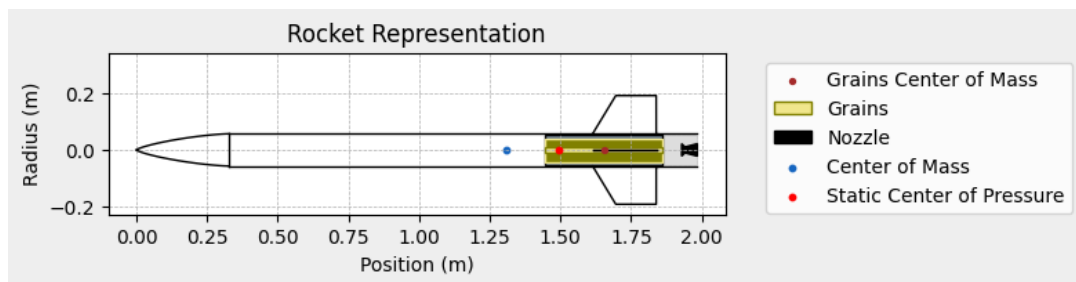
– Radial inertia as a function of time



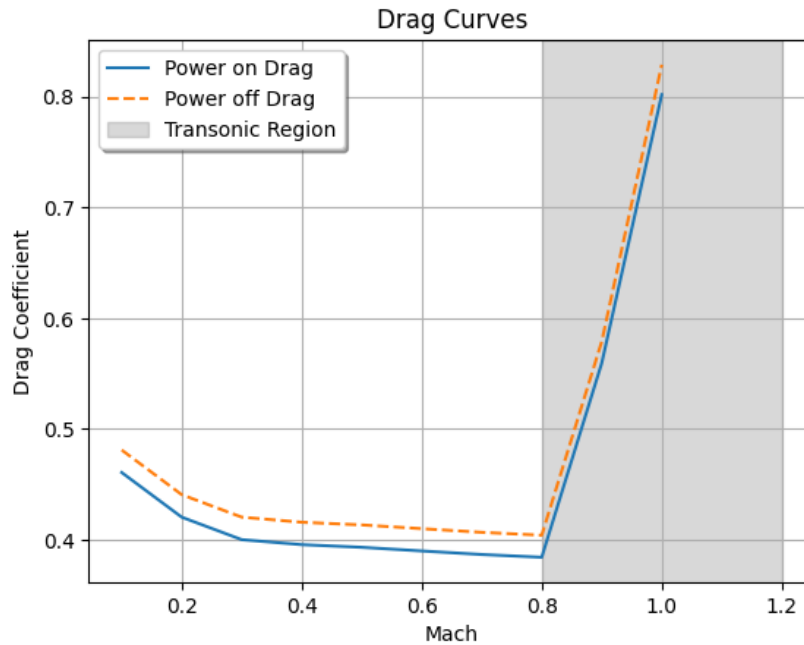
– Static margin as a function of time



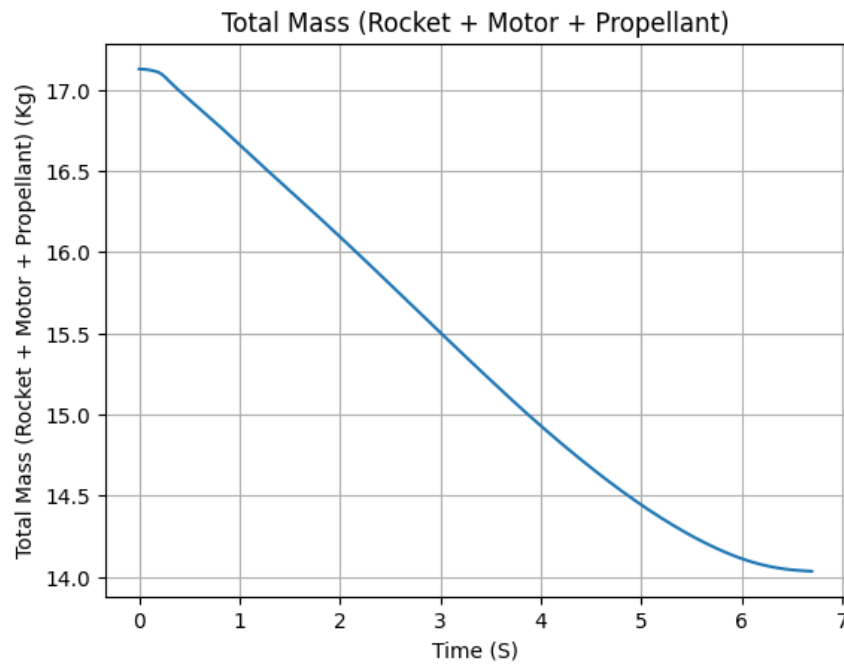
- Initial center of gravity position: 1.30 m, with respect to the tip of the nose cone
- Initial center of pressure position: 1.49 m, with respect to the tip of the nose cone



- CD as a function of Mach



- Masa inicial del cohete total(dry mass + propelente): 17.12 kg
- Masa del cohete en función del tiempo



NOTE: It is only shown on the graph until the engine shuts down; from that moment on it will remain constant with a value of: 14.0359 kg.

2.2 Analysis with a heading angle such that it maximizes the apogee

2.2.1 Reasoning for maximizing the apogee

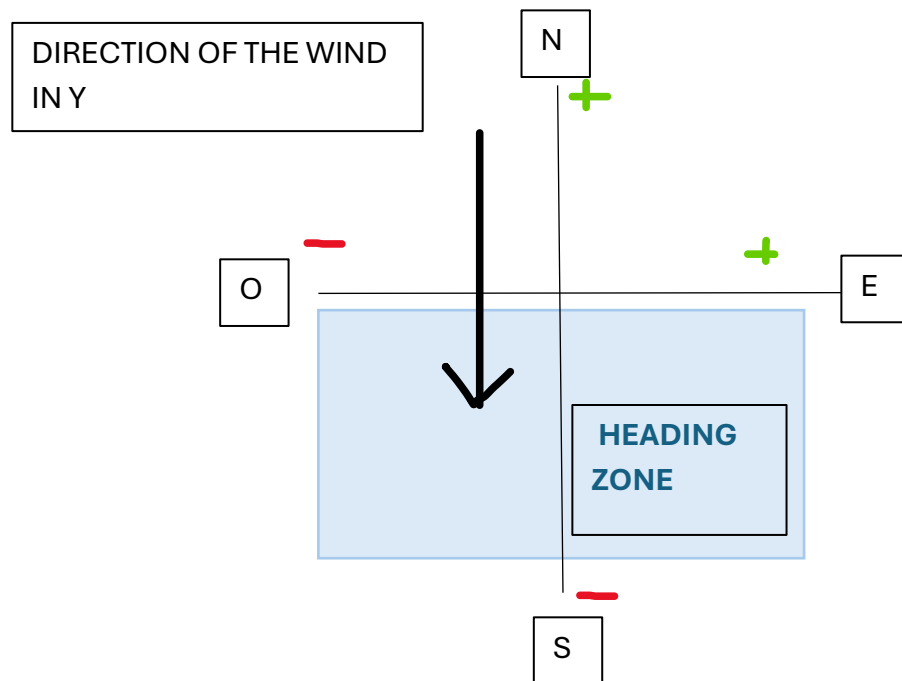
To maximize the apogee, we must select the heading that is most favored by the wind direction; in the case of my simulation, the wind components are:

Y: -5.76 m/s

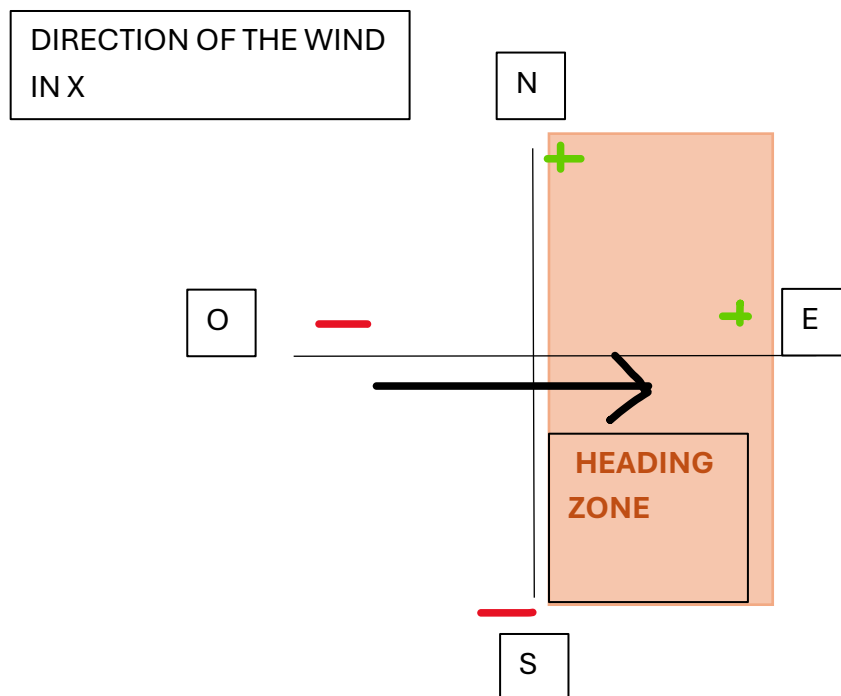
X: 4.39 m/s

Due to the flight dynamics of the rocket when it is loaded, it turns in the direction of the wind due to the moment generated by the aerodynamic elements. Therefore, to obtain the maximum apogee (the highest point of the trajectory), we must orient the rail slightly in favor of the wind direction.

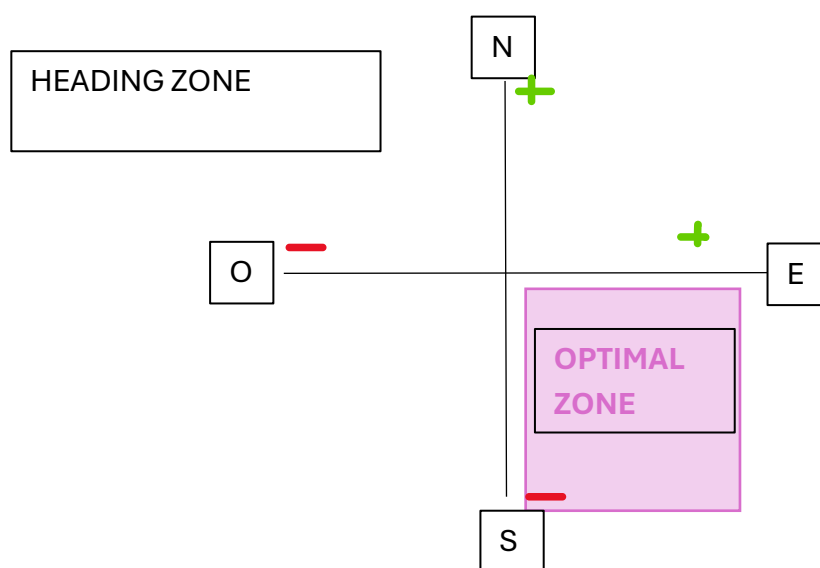
Consequently, since the wind on the Y axis goes from north to south (it has a negative value), the heading will have to be in the II or III quadrant, as shown in the representation:



On the other hand, the wind on the x-axis has a positive direction from west to east; therefore, in this case, since the wind is positive, it follows this direction. As a consequence, the heading must be in the I or II quadrant. We can visualize this deduction as follows:



Therefore, combining both, we have that the heading (angle) must be in the II quadrant.

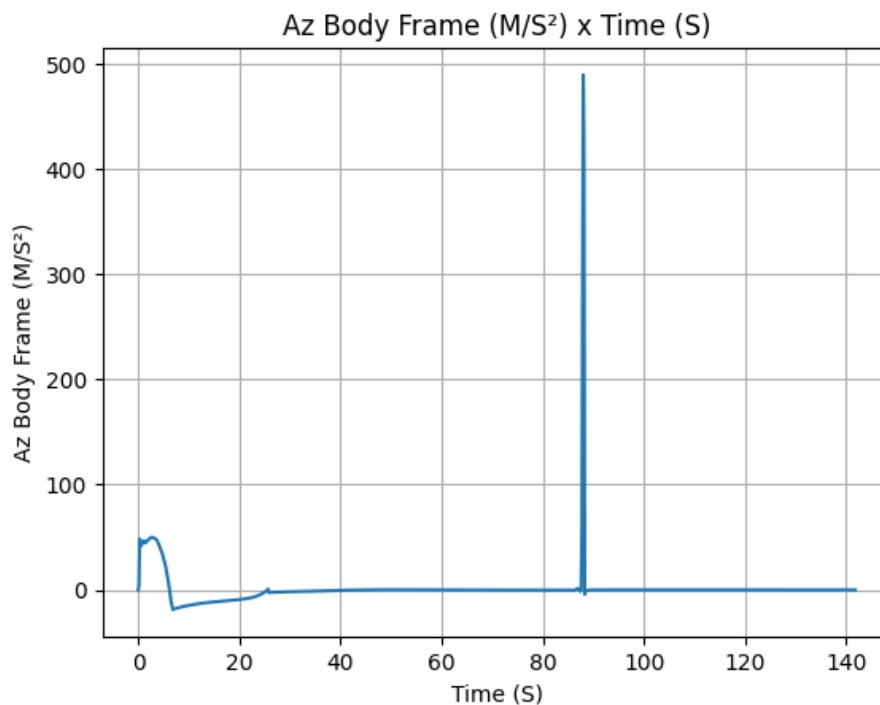


Furthermore, as the vertical component is greater than the horizontal one, its sine in absolute value is greater than the cosine. So the angle will point more towards the south than towards the east.

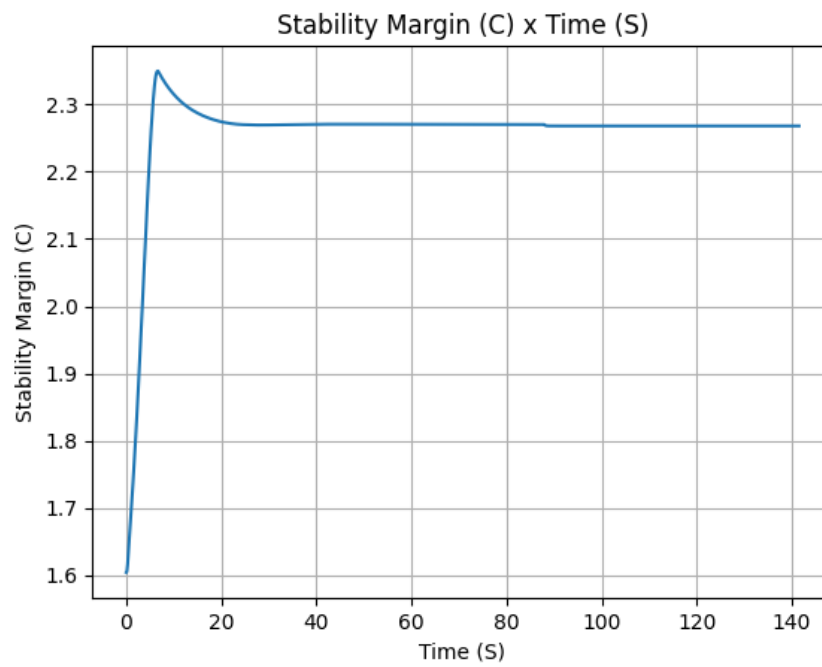
The heading is measured with respect to the positive y-axis (North), so angles in the II quadrant are between 90° and 180° . Since we know its sine is greater in absolute value, we can estimate that it will exceed 135° . Therefore, an ideal angle would be comprised between 140° and 150° . With 145° being approximately the angle that maximizes the apogee.

2.2.2 Parameters obtained with said heading

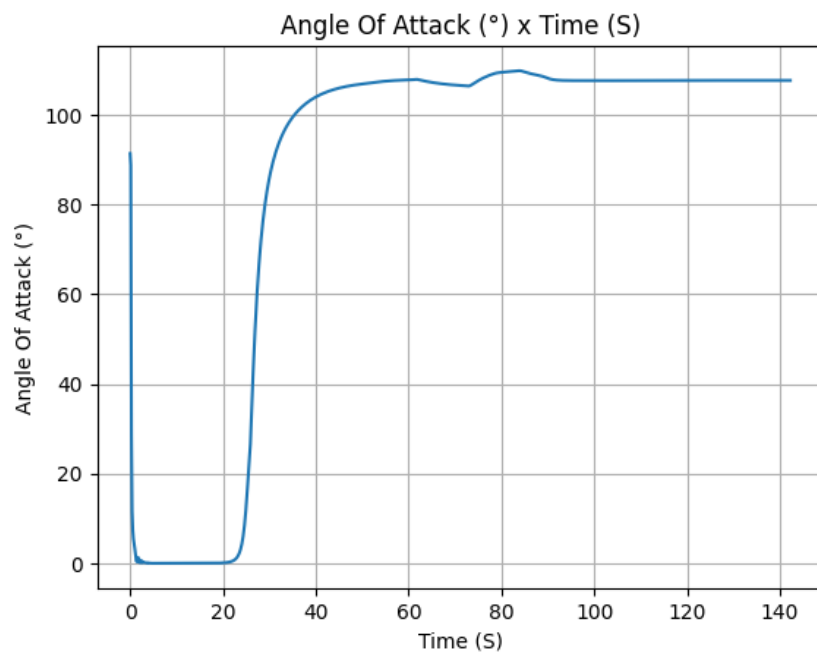
- Rail exit velocity: 32.39 m/s
- Time to clear the rail: 0.933 s
- Time to reach apogee: 25.52 s
- Maximum velocity (magnitude): 235.28 m/s
- Longitudinal acceleration as a function of time: (dry mass)



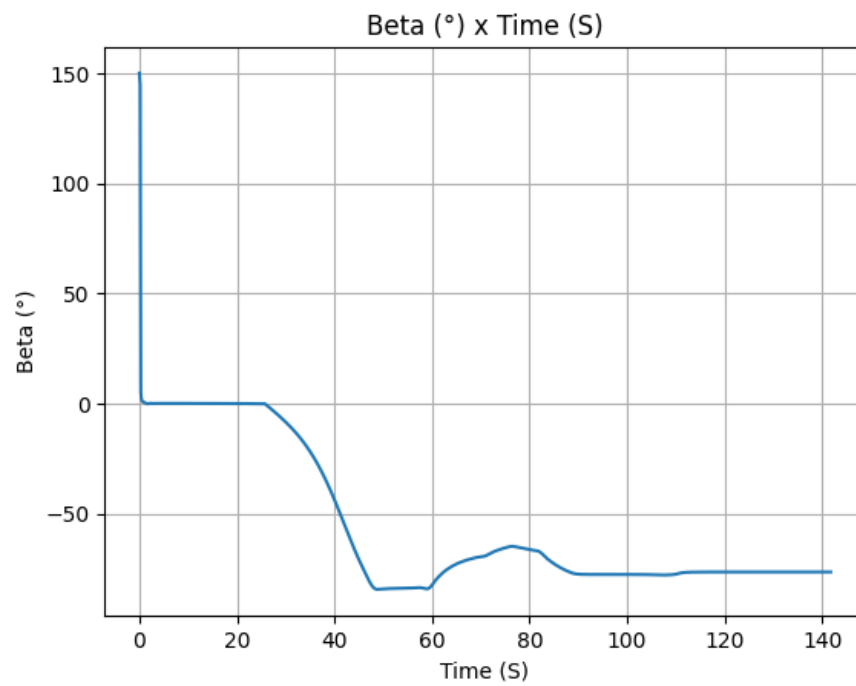
- Maximum acceleration (in magnitude) in the longitudinal axis: 50.48 m/s²
- Stability margin as a function of time



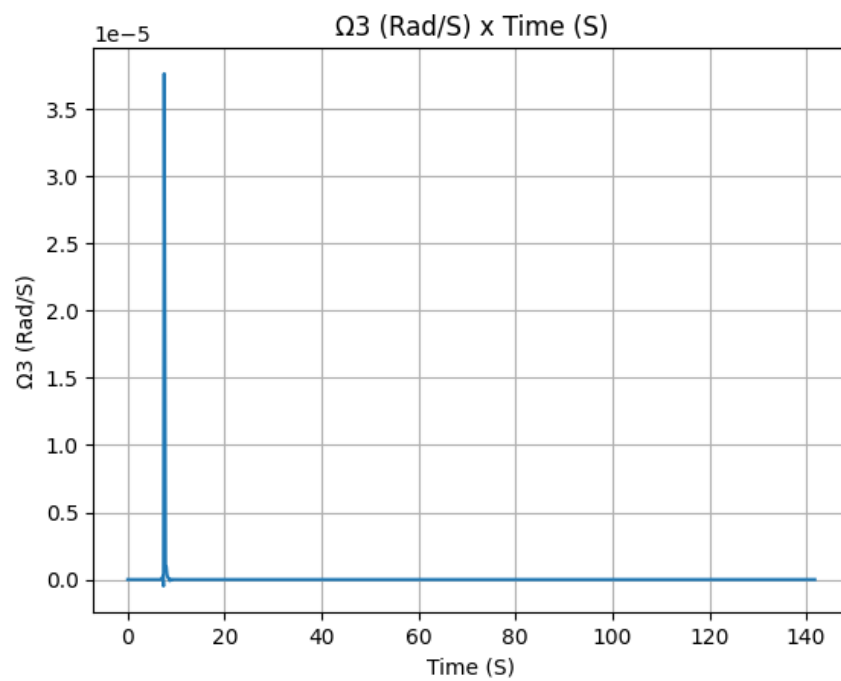
- Stability margin upon exiting the rail: 1.68
- Angle of attack as a function of time



- Sideslip angle as a function of time

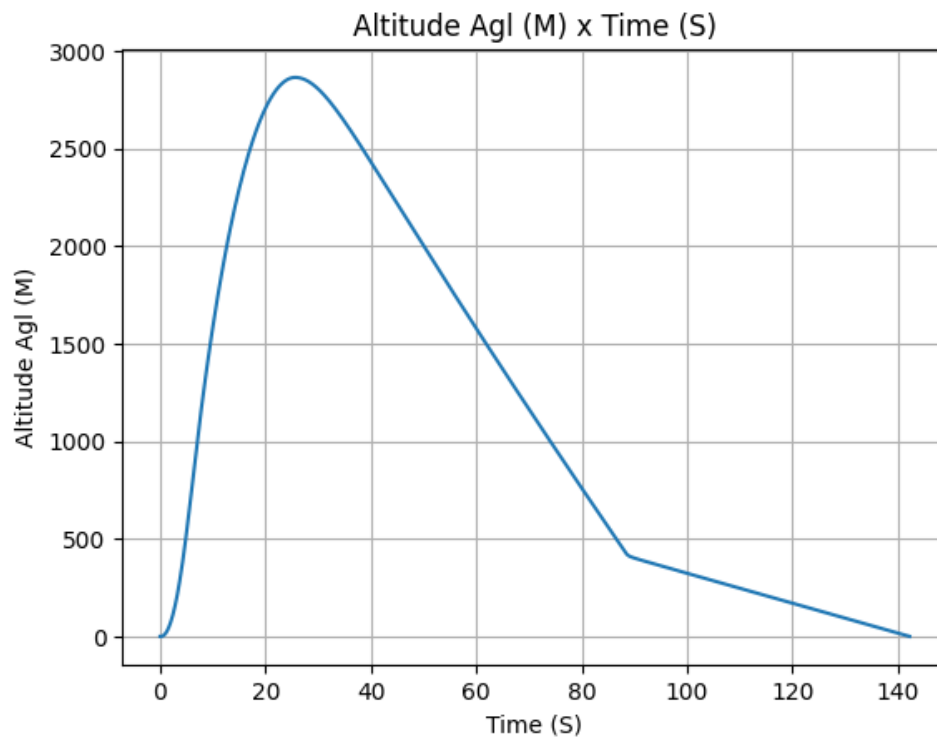


- Roll velocity as a function of time



- (x,y) coordinates of the apogee: (-306.62, 359.76) m
- (x,y) coordinates of impact: (-147.73, -1215.59) m

- Fall downrange: 1224.54 m (two-dimensional vector norm, ignoring possible altitude change, running from the launch site (0,0) to the point of impact)
- Altitude as a function of time



- Apogee: 2837.545 m